

A PRACTITIONER'S FIELD GUIDE

Detection Engineering in Practice

Building, running, and tuning a YARA-based SOC detector

Most security writing stops at the rule. This book follows a single real detector all the way through — from the first YARA string to the alert a tired analyst has to judge at 3 a.m. Every example is drawn from one working, published codebase, including the bugs found while building it.

Larry Allen

<https://github.com/gocar112/security-suite-dashboard/tree/main>

First edition — September 2026

Contents

Preface.....	6
Who this is for.....	6
How to read it.....	6
On the word "practice".....	7
The Detection Engineer's Job.....	9
The detection lifecycle.....	9
Three ways a detection fails.....	9
The missed detection.....	10
The false positive.....	10
The unactionable alert.....	10
Detection versus prevention.....	10
Where this book sits.....	10
The File Drop as an Attack Surface.....	12
Why directories are where this starts.....	12
The five things that land in an uploads folder.....	12
1. Web shells.....	12
2. Ransomware notes and staging.....	12
3. Offensive tooling.....	13
4. Fileless payloads written to disk anyway.....	13
5. Social engineering artifacts.....	13
The threat model, stated.....	13
The one question to ask before every rule.....	14
YARA, Precisely.....	16
Anatomy of a rule.....	16
Strings.....	16
Text strings.....	16
Hexadecimal strings.....	17
Regular expressions.....	17
Conditions.....	17
Counting and sets.....	18
Positions and size.....	18
Modules.....	18
Namespaces, tags, and organisation.....	19
Compiling and matching in Python.....	19
The timeout argument nobody passes.....	19

Rules That Survive Contact with Reality	21
The single-string trap.....	21
Two-factor conditions.....	21
The same shape, three more times.....	22
Anchoring and bounding.....	23
Indicator rules and their honest limits.....	23
Behaviour rules: the strongest kind.....	24
Metadata is a contract.....	24
Anatomy of a Detector	27
Starting from a script.....	27
Decomposition.....	27
The engine: compile once, scan many.....	28
Enrichment at scan time.....	29
Recording the match, not just the verdict.....	29
The store: durability, history, and a live feed.....	30
Why the ring buffer matters.....	31
Choosing not to use a framework.....	31
The Race Conditions Nobody Warns You About	32
Bug one: the half-written file.....	32
The fix: settle detection.....	32
Bug two: the self-scan feedback loop.....	33
The fix, and the principle.....	33
Bug three: the file that vanishes mid-scan.....	34
The fix: distinguish absence from failure.....	34
Bug four: forgetting to forget.....	35
The baseline decision.....	35
Telemetry, Time, and Correlation	36
A bug worth studying in detail.....	36
Timestamps are the whole problem.....	37
Syslog, and the missing year.....	37
The Windows Security log.....	38
Permission as a first-class state.....	38
Choosing a window.....	39
What a correlated alert actually claims.....	39
Severity, Triage, and the Economics of Attention	42
A severity scale that means something.....	42
Severity belongs to the rule.....	43

Triage states	43
The only KPI that matters	44
Designing the alert for the reader	44
Measuring and Tuning a Ruleset	45
Precision and recall, in the terms that matter	45
Testing detection content	45
Test the negative cases	46
The tuning loop	46
Hot reload makes tuning cheap	46
Performance as a detection property	47
Running It for Real	48
Privileges: ask for the minimum, degrade honestly	48
The network boundary	48
Feeding a SIEM	49
What actually breaks	49
Extending it	49
Closing	50
The Complete Ruleset	52
demo.yar — pipeline verification	52
webshell.yar — server-side backdoors	52
windows_threats.yar — host tradecraft	52
HTTP API Reference	54
Event shape	54
Configuration Reference	56
Command Line and Recipes	58
Invocation	58
Verifying an installation	58
Querying the log	58
Further reading	59

FRONT MATTER

Preface

This book has one detector in it. Not a survey of twelve tools, not a vendor comparison — one small, complete, working sensor, followed from the first line of a YARA rule to the moment an analyst decides whether an alert was worth their attention.

There is a lot of security writing that explains what a detection rule *is*. There is much less that explains what happens to that rule once it meets a real filesystem, a real antivirus product, a real log with missing timestamps, and a real person who has forty other alerts open. The gap between those two things is where detection engineering actually lives, and it is almost entirely made of unglamorous problems: files that are still being written when you scan them, logs you do not have permission to read, a severity field nobody agreed on, and an alert queue that grows faster than anyone can clear it.

The running example throughout is **Security Suite**, a YARA-backed file detector with a live dashboard, published at <https://github.com/gocar112/security-suite-dashboard/tree/main>. It is small enough to read in an afternoon — roughly 1,400 lines of Python and 1,000 of front-end — and complete enough to have hit every class of problem this book describes. Where the text says "this broke", it broke. Where it says "the fix was", you can read the fix.

Who this is for

You should be comfortable reading Python and willing to look at a regular expression without flinching. You do not need prior YARA experience; Chapter 3 builds it from nothing. You do not need to have worked in a SOC, though if you have, several chapters will read as confirmation rather than instruction.

The book is aimed at three people in particular. The **analyst** who is tired of tuning someone else's rules and wants to understand the machinery underneath them. The **developer** who has been handed "add malware scanning to the upload endpoint" and correctly suspects it is harder than it sounds. And the **student or career-changer** who wants a portfolio project that demonstrates judgment rather than just syntax.

How to read it

Part I frames the discipline and the threat model. Part II is about writing detection content — YARA specifically, but the reasoning generalizes to Sigma, Suricata, and anything else with a condition block. Part III builds the sensor and is where most of the hard-won material lives: the race conditions, the timestamp bug, the feedback loop that made the detector alert on itself. Part IV is about operating the thing once it exists, which is the part most projects skip and most deployments fail on.

Chapters are meant to be read in order, but Part III and Part IV stand alone well enough if you already write rules for a living. The appendices are reference material: the full ruleset, the HTTP API, every configuration key, and the command line.

A NOTE ON THE CODE

Every code listing is real, taken from the published repository rather than written for the page. Line-level details may drift as the project evolves; the reasoning will not. Clone it, break it, and watch what the dashboard says.

On the word "practice"

The title is not decoration. A detection that works on your laptop against a file you crafted yourself is a demonstration. A detection that works on a machine you do not control, against a file someone else crafted, while an antivirus product is actively deleting your evidence, is a practice. This book is about the second kind, and it gets there by making the first kind fail in instructive ways.

PART ONE

The Discipline

What detection engineering is, what it is responsible for, and the specific corner of it this book lives in.

CHAPTER ONE

The Detection Engineer's Job

Detection engineering is the practice of turning knowledge about attacker behaviour into automated signal that a human can act on — and then taking responsibility for what that signal does to the human.

The second half of that sentence is the part that gets dropped. It is straightforward to write a rule that fires on something bad. It is considerably harder to write one that fires on something bad, does not fire on the nine thousand benign files that look vaguely similar, arrives with enough context that an analyst can judge it in under a minute, and keeps behaving that way six months later when the environment has changed and nobody remembers why the rule exists.

The detection lifecycle

Detection work runs in a loop, and every stage of it can fail independently:

1. **Hypothesis.** A belief about attacker behaviour. "An attacker who gets file-upload access will drop a web shell." This comes from threat intelligence, incident history, red team findings, or reasoning about your own architecture.
2. **Instrumentation.** Can you even see the thing your hypothesis is about? If files land in a directory no process watches, no rule will ever fire. Visibility precedes detection and is usually the real bottleneck.
3. **Detection content.** The rule itself — the YARA condition, the Sigma correlation, the query. This is the part everyone writes about.
4. **Enrichment.** The context attached to a hit: file hash, who uploaded it, what else happened around the same time. An alert without enrichment is a question, not an answer.
5. **Triage.** A human decides: real, not real, or needs investigation. Everything upstream exists to make this decision fast and correct.
6. **Tuning.** What you learn in triage feeds back into the rule. A detection that is never tuned is a detection that is decaying.

Security Suite implements all six, deliberately and visibly. The hypothesis is encoded in `rules/`. The instrumentation is the directory monitor. The content is YARA. The enrichment is the SHA-256, the entropy, the matched strings, and the correlated authentication failures. The triage is four buttons in a web UI. The tuning is a rule file you edit and hot-reload. Being able to point at each stage in a system small enough to hold in your head is the entire reason this book uses one codebase instead of many.

Three ways a detection fails

It helps to be precise about failure, because the three modes have different costs and pull in different directions.

The missed detection

The attacker does the thing, and nothing fires. This is the failure everyone fears and the hardest to measure, because by definition you do not have a record of it. You discover false negatives through incident response, red teaming, or purple team exercises — never through your alert queue, which is exactly the trap.

The false positive

Something benign fires the rule. Each one is individually cheap and collectively ruinous. A rule with a 2% false positive rate sounds excellent until you run it against 50,000 files a day and it generates a thousand alerts nobody will ever read.

The unactionable alert

This is the underrated one. The rule fired, it was even correct, and the analyst still cannot do anything with it — because it says `MALWARE_DETECTED` and nothing else. No file path, no hash, no indication of what matched or why. It is technically a true positive and operationally worthless.

Notice that these trade against each other. Broadening a rule to reduce false negatives increases false positives. Narrowing it does the reverse. Adding enrichment costs scan time and complexity. Every detection is a position on those axes, and a good detection engineer can state where their rule sits and why.

THE ASYMMETRY WORTH REMEMBERING

A false negative costs you one incident. A high false positive rate costs you every incident, because it trains the analyst to stop looking. Alert fatigue is not a morale problem; it is a detection failure with a delayed fuse.

Detection versus prevention

Security Suite detects. It does not quarantine, delete, block, or move anything. That is a deliberate architectural choice, and it is worth understanding why.

Prevention has a failure mode detection does not: when prevention is wrong, it breaks production. An antivirus product that false-positives on your deployment artifact does not generate an alert somebody can dismiss — it deletes your build and takes the service down. The blast radius of a wrong preventive action is enormous, which is why preventive controls must be held to a far higher precision standard than detective ones.

Detection is allowed to be wrong. That freedom is what lets a detection engineer ship a rule at 80% precision, watch what it catches, and tune it — a process that would be unacceptable if every firing deleted a customer file. Keep them separate, and be suspicious of any tool that quietly blurs the line.

Where this book sits

Detection spans network, endpoint, identity, cloud control plane, and application layers. This book lives almost entirely in one narrow place: **files arriving on disk**. That narrowness is a feature. File-based detection is the oldest and best-understood corner of the discipline, it has a mature tool in YARA, and every hard problem in the wider field — visibility gaps, timestamp handling, severity taxonomy, alert economics — shows up there in miniature.

What you learn here about correlating a file event with an authentication event transfers directly to correlating a process launch with a DNS query. The plumbing changes; the reasoning does not.

CHAPTER TWO

The File Drop as an Attack Surface

Before writing a single rule, you need a threat model: who is putting files where, and what do they want those files to do?

A rule written without a threat model is a rule written against your imagination. It will catch what you happened to think of and miss what you did not, and you will have no way of knowing which is which. So this chapter establishes the specific threat model the case study defends, and the reasoning applies to whatever directory you end up watching.

Why directories are where this starts

Almost every system that accepts input from the outside world eventually writes that input to disk. Upload forms, FTP drop boxes, email attachment processors, document conversion pipelines, CI artifact stores, network shares, backup landing zones. Each one is a place where content controlled by someone else comes to rest inside your trust boundary.

That makes watched directories unusually high-value for detection. The attacker has to materialize something — a payload, a tool, a script — and for a moment it sits still, unencrypted, in a place you control. Network detection sees a stream you may not be able to decrypt. Process detection sees behaviour only after execution begins. File detection gets the artifact itself, at rest, before it runs.

The five things that land in an uploads folder

1. Web shells

The classic. An attacker who finds an upload form that does not validate file types drops a small script — PHP, ASPX, JSP — into a directory the web server will execute. From then on they have command execution, and it looks like ordinary web traffic.

Web shells are an excellent detection target because they have an irreducible structure. To be useful, a web shell must take input from the request and pass it to something that executes. It can be obfuscated in a thousand ways, but those two elements have to be present, and that pairing is what a good rule keys on. Chapter 4 develops exactly this.

2. Ransomware notes and staging

By the time a ransom note appears, encryption is usually well underway — so detecting the note is late. It is still worth doing, for two reasons: it is unambiguous (almost nothing benign says "your files have been encrypted" next to a payment address), and it gives you a precise timestamp for incident reconstruction. Earlier signals — shadow

copy deletion, security tooling being disabled — are more valuable, and the case study covers both.

3. Offensive tooling

Credential dumpers, lateral movement utilities, enumeration scripts. These arrive after initial access, when the attacker is expanding. They are often well-known public tools with well-known strings, which makes them easy to detect and correspondingly easy for a competent adversary to modify. Detecting the unmodified version is still worth doing: most intrusions are not run by competent adversaries.

4. Fileless payloads written to disk anyway

The industry says "fileless" and then writes an encoded PowerShell command into a scheduled task definition, a log file, a registry export, or a batch script. The payload does not persist as an executable, but the *command* frequently does persist as text somewhere on disk — which is why the case study's ruleset looks for encoded command patterns and download cradles in plain text files, not only in binaries.

5. Social engineering artifacts

Double extensions (`invoice.pdf.exe`), documents with macro lures, LNK files pointing at scripts. These target the user rather than the system, and they show up in mail stores and download folders more than in server upload directories.

The threat model, stated

Putting that together, here is what Security Suite is built to defend, written plainly enough to argue with:

Element	Assumption	Consequence for design
Adversary	Opportunistic to moderately skilled. Uses public tooling, may lightly obfuscate, is not writing a bespoke packer for this target.	String and pattern matching is worth doing. Heavy anti-evasion machinery is not the first priority.
Access	Can write files into a watched directory, by upload, transfer, or compromised account.	The directory is the detection point. Files are untrusted on arrival.
Goal	Execute code, persist, collect credentials, or extort.	Rules target execution primitives and extortion artifacts, not file types.
Defender	One or a few people, not a 24/7 tier-one team.	Alert volume must stay low. Every alert must be judgeable in under a minute.
Environment	Windows-first, Linux-capable. Antivirus is present and active.	Telemetry reads the Windows Security log. The scanner must survive AV deleting its evidence.

WHAT THIS MODEL EXCLUDES

A nation-state adversary writing a custom loader for your specific environment will not be caught by string matching, and pretending otherwise is how security programmes waste years. This detector raises the cost of the common case. That is a real and worthwhile goal — it is simply not the same goal as defeating a determined, well-resourced attacker, and conflating the two produces bad architecture and worse expectations.

The one question to ask before every rule

For each rule you write, you should be able to answer: **what does a true positive look like, and what will I do about it?**

If the answer to the first half is vague, the rule is a guess. If the answer to the second half is "nothing, really", the rule is noise you have not yet noticed. Detection content that nobody would act on should not exist; it dilutes attention away from content that matters. This is the single most effective filter for keeping a ruleset healthy, and the easiest to skip.

PART TWO

Writing Detections

YARA from first principles, and the difference between a rule that matches and a rule that holds up.

CHAPTER THREE

YARA, Precisely

YARA is a pattern matching language for classifying files. This chapter builds it from nothing, using only constructs the case study actually uses.

YARA has a reputation as "grep for malware", which undersells it badly. Grep asks whether a pattern appears. YARA asks whether a *combination* of patterns appears, in particular positions, in a file of a particular size, and it evaluates that question fast enough to run against every file on a filesystem.

Anatomy of a rule

Every YARA rule has the same three-part shape. Here is one from the case study, complete:

```
rule PowerShell_Encoded_Command : suspicious
{
    meta:
        description = "PowerShell launched with a base64 encoded command block"
        severity    = "high"
        author      = "security-suite"
    strings:
        $ps = "powershell" nocase
        $e1 = "-encodedcommand" nocase
        $e2 = " -enc " nocase
        $e3 = "-e JAB" nocase
        $e4 = "FromBase64String" nocase
    condition:
        $ps and any of ($e*)
}
```

Reading it top to bottom: `rule` names it, `: suspicious` attaches a tag, `meta:` holds arbitrary key-value pairs that do not affect matching, `strings:` declares patterns, and `condition:` is the boolean expression that decides. Only the condition determines a match. Metadata is for humans and for downstream tooling — which, as Chapter 8 shows, is how severity gets into the dashboard.

Strings

YARA has three kinds of string, and knowing when to reach for each is most of the craft.

Text strings

A literal sequence of characters, optionally decorated with modifiers:

```
$a = "sekurlsa::logonpasswords" nocase
$b = "MZ" ascii
$c = "Invoke-Expression" nocase wide
$d = "eval" fullword
```

Modifier	Effect
<code>nocase</code>	Case-insensitive. Use it by default for anything human-typed — command lines, script keywords, note text. Costs a little performance, prevents a large class of trivial evasion.
<code>wide</code>	Matches UTF-16LE encoding, where each character is followed by a null byte. Windows binaries store most strings this way. A rule looking for a Windows API name without <code>wide</code> will silently miss.
<code>ascii</code>	Matches single-byte encoding. This is the default, but becomes explicit and necessary the moment you add <code>wide</code> — <code>wide</code> alone replaces ASCII matching rather than adding to it.
<code>fullword</code>	Requires non-alphanumeric boundaries. <code>"eval"</code> <code>fullword</code> matches <code>eval(</code> but not <code>medieval</code> . Essential for short strings.
<code>xor</code>	Matches the string XOR-encoded against single-byte keys. Catches lazy obfuscation; expands the search space, so use deliberately.
<code>base64</code>	Matches the base64 encodings of the string. Useful against encoded configuration blobs.

THE WIDE/ASCII TRAP

This is the most common beginner mistake in YARA, and it fails silently. Writing `$a = "CreateRemoteThread"` against a Windows PE will often not match, because the string is stored as UTF-16. Writing `$a = "CreateRemoteThread" wide` will then miss any ASCII copy. What you almost always want is `wide ascii` — both. The rule compiles and runs either way, which is precisely why the mistake survives.

Hexadecimal strings

For binary content, expressed as bytes with wildcards:

```
$mz    = { 4D 5A }           // exact: the PE magic bytes "MZ"
$stub  = { 4D 5A ?? ?? 00 00 } // ?? = any single byte
$range = { E8 [4-8] 5D C3 }   // [n-m] = between n and m arbitrary
bytes
$alt   = { 6A 40 ( 68 | 6A ) } // ( | ) = alternatives
```

Hex strings are how you match file structure rather than file content: headers, magic bytes, instruction sequences, embedded structures. The case study uses exactly one, `{ 4D 5A }`, to identify PE files — and pairs it with a position anchor, which is the subject of the next section.

Regular expressions

YARA supports regex between forward slashes: `/https?:\\/[a-z0-9.-]+\\.onion/ nocase`. They are powerful and considerably slower than text or hex strings. The working rule is to reach for regex only when the alternatives genuinely cannot express the pattern — a handful of text strings combined in the condition usually can, and will run far faster.

Conditions

The condition is a boolean expression, and it is where a rule becomes precise rather than merely present.

Counting and sets

```
condition:
    $a                                // this string appears at least once
    any of them                       // at least one declared string
    all of them                       // every declared string
    2 of ($a, $b, $c)                // at least two of these three
    any of ($e*)                     // at least one string named $e-something
    #a > 3                            // $a appears more than three times
    #a == 1                          // exactly once
```

The `any of ($prefix*)` form is the workhorse. It lets you declare a family of related indicators and require one, which is how you express "any of these several ways of spelling the same idea" without a wall of `or`.

Positions and size

```
condition:
    $mz at 0                          // string occurs exactly at offset 0
    $sig in (0..1024)                 // occurs somewhere in the first KB
    @a[1] < @b[1]                     // first $a occurs before first $b
    filesize < 20MB                   // file size bound
    uint16(0) == 0x5A4D                // read two bytes at offset 0 as an integer
```

Position anchoring is a precision multiplier and an underused one. `$mz at 0` means "this file begins with MZ", which is meaningfully different from "the letters MZ appear somewhere in this file" — the latter matches any text document containing the word *amazing*. Free precision, one keyword.

Modules

YARA ships modules that expose computed properties. The case study imports one:

```
import "math"

rule High_Entropy_Executable : suspicious
{
    meta:
        description = "PE file whose body looks packed or encrypted"
        severity    = "medium"
    strings:
        $mz = { 4D 5A }
    condition:
        $mz at 0 and math.entropy(0, filesize) > 7.2 and filesize < 20MB
}
```

`math.entropy` computes Shannon entropy over a byte range. The `pe` module goes much further — sections, imports, exports, signatures, resources — and is worth learning once you are matching on binaries in earnest. Note that `import` must appear at the top

of the file, before any rule; forgetting this is a compile error that reads more confusingly than it should.

Namespaces, tags, and organisation

When you compile several rule files together, each becomes a **namespace**. Rule names must be unique within a namespace, not globally, which means two teams can both have a rule called `Suspicious_Archive` without collision. The case study compiles one namespace per file:

```
sources = {}
for path in rule_files:
    namespace = path.stem          # rules/*.yar
    sources[namespace] = path.read_text()  # "demo", "webshell", "windows_threats"

compiled = yara.compile(sources=sources)
```

Tags (the `:` `suspicious` after a rule name) are a second, orthogonal axis — a rule can carry several. They are returned with every match, which makes them useful for routing: send everything tagged `ransomware` to the on-call phone, everything tagged `test` to a log nobody pages on.

Compiling and matching in Python

The case study uses `yara-python` rather than shelling out to the `yara` binary. This matters more than it sounds: compiling rules once and holding the compiled object in memory removes both process spawn overhead and recompilation from every single scan.

```
import yara

rules = yara.compile(filepath="rules/webshell.yar")      # one file
rules = yara.compile(sources={"ns": source_text})        # from strings
rules = yara.compile(filepaths={"ns": "rules/a.yar"})    # several files

matches = rules.match(filepath="suspect.bin", timeout=60)
matches = rules.match(data=buffer)                      # in-memory bytes

for m in matches:
    print(m.rule, m.namespace, m.tags, m.meta)
    for s in m.strings:
        # StringMatch
        for i in s.instances:
            # StringMatchInstance
            print(s.identifier, hex(i.offset), i.matched_data)
```

API VERSION NOTE

The nested strings / instances structure above is the modern `yara-python` API (4.3 and later). Older code returns flat `(offset, identifier, data)` tuples, and a great deal of tutorial material still shows that form. If a match-iteration example fails with an attribute error, an API generation mismatch is the first thing to check. The case study targets 4.5.

The timeout argument nobody passes

`rules.match(filepath=..., timeout=60)` bounds how long matching may run. It is optional and routinely omitted, which is a mistake in any long-running scanner: a pathological file combined with an expensive regex can hang a scan indefinitely, and a monitor blocked forever on one file is a monitor that has silently stopped protecting you. Setting a timeout converts an invisible hang into a visible error.

CHAPTER FOUR

Rules That Survive Contact with Reality

A rule that matches your test file is not a detection. It becomes one when it still behaves after meeting a hundred thousand files you did not choose.

This chapter is about the difference. It works through the case study's ruleset, rule by rule, and names the technique each one demonstrates.

The single-string trap

The obvious first web shell rule is the wrong one:

```
rule Bad_Webshell
{
    strings:
        $a = "eval(" nocase
    condition:
        $a
}
```

This fires on every JavaScript library that has ever shipped a polyfill, every minifier output, every piece of documentation about `eval`, and a large fraction of legitimate PHP. It is not a detection; it is a fuzzy `grep` that produces alerts. The rule is not *wrong* about `eval` being interesting — it is wrong about `eval` alone being sufficient.

Two-factor conditions

The fix is to require two independent elements that are individually common and jointly rare. Here is the case study's actual rule:

```
rule PHP_Webshell_Eval_Input : webshell
{
    meta:
        description = "PHP one-liner passing request data straight to an evaluator"
        severity = "critical"
        author = "security-suite"
    strings:
        $php = "<?php"

        $e1 = "eval(" nocase
        $e2 = "assert(" nocase
        $e3 = "preg_replace" nocase

        $src1 = "$_POST" nocase
        $src2 = "$_GET" nocase
        $src3 = "$_REQUEST" nocase
        $src4 = "php://input" nocase
    condition:
        $php and any of ($e*) and any of ($src*)
}
```

```
| }
```

Read the condition as a sentence: *this is a PHP file **and** it contains an evaluation primitive **and** it contains a source of attacker-controlled input*. Each clause on its own is worthless. Together they describe the irreducible structure of a web shell identified in Chapter 2 — untrusted input reaching an execution primitive.

Three properties make this durable. It is **structural**, keying on what a web shell must do rather than how a particular one is written. It is **resilient to variation**, because the `$e*` and `$src*` families absorb alternatives without touching the condition. And it **fails safely**: a PHP file with `eval` but no request input does not fire, which is correct, because that file is a template engine.

THE GENERALISABLE PATTERN

Capability plus context. Find the two things the attacker cannot avoid doing, and require both. This survives obfuscation better than any single-string rule, because obfuscation changes spelling while structure stays fixed.

It is also the reason to prefer several short strings in families over one long clever one. Long strings are brittle — one changed byte and you have a false negative you will never hear about.

The same shape, three more times

Once you have the pattern, the rest of the web shell ruleset writes itself. The ASPX variant pairs a page directive with a request accessor and an execution primitive:

```
rule ASPX_Webshell : webshell
{
  strings:
    $page = "<%@ Page" nocase
    $a1 = "Request.Item" nocase
    $a2 = "Request.Form" nocase
    $b1 = "System.Diagnostics.Process" nocase
    $b2 = "CreateObject" nocase
    $b3 = "ExecuteReader" nocase
  condition:
    $page and any of ($a*) and any of ($b*)
}
```

And the JSP variant, which is tighter still because the Java idiom for running a command is so specific:

```
rule JSP_Webshell : webshell
{
  strings:
    $jsp = "<%"
    $req = "request.getParameter" nocase
    $exec1 = "Runtime.getRuntime().exec" nocase
    $exec2 = "ProcessBuilder" nocase
  condition:
    $jsp and $req and any of ($exec*)
}
```

Three files, one idea, consistently applied. That consistency is worth as much as the individual rules: when every rule in a set is built the same way, the set is far easier to review, tune, and hand to someone else.

Anchoring and bounding

The entropy rule from Chapter 3 demonstrates two more precision techniques in one line:

```
condition:
    $mz at 0 and math.entropy(0, filesize) > 7.2 and filesize < 20MB
```

`$mz at 0` anchors position — the file *starts* with MZ, so it is a PE, not a text file mentioning MZ. `filesize < 20MB` bounds cost, because entropy over a 4 GB file is expensive and rarely informative. Between them they turn a vague heuristic into something you can afford to run on every file.

The 7.2 threshold deserves comment. English text sits around 4.5 bits per byte, normal executable code around 6.0 to 6.5, and compressed or encrypted data pushes toward the theoretical maximum of 8.0. The 7.2 line catches packed and encrypted content while leaving ordinary binaries alone. It also catches every legitimate installer, self-extracting archive, and anything else compressed — which is why this rule is `severity = "medium"` and not higher. It is a *hint*, and the metadata says so.

Indicator rules and their honest limits

Some rules are just lists of known-bad strings:

```
rule Credential_Dumper_Indicators : malware
{
    meta:
        description = "Strings associated with LSASS credential dumping tools"
        severity = "critical"
    strings:
        $a1 = "sekurlsa::logonpasswords" nocase
        $a2 = "privilege::debug" nocase
        $a3 = "lsadump::sam" nocase
        $a4 = "mimikatz" nocase
        $a5 = "gentilkiwi" nocase
    condition:
        any of them
}
```

This is a fundamentally weaker rule than the web shell ones, and it is important to be clear-eyed about why. It matches *names*, not *behaviour*. Renaming defeats it. A recompiled variant with the command strings changed walks straight past.

So why ship it? Because unmodified public tooling remains extremely common, because the strings are distinctive enough that false positives are rare, and because it costs nothing to run. It is a cheap net for the careless majority. What it must not be is your

only control against credential theft, and a ruleset that consists entirely of rules like this is far weaker than its rule count suggests.

A CAVEAT ON INDICATOR RULES

This rule will fire on security documentation, training material, detection rules themselves, and blog posts about credential dumping. In a SOC folder full of analysis notes it is nearly useless; in an FTP upload directory it is excellent. The same rule has different precision in different places — which is an argument for scoping rulesets to directories, not deploying one global set everywhere.

Behaviour rules: the strongest kind

Compare the dumper rule with this one:

```
rule Defense_Evasion_Commands : suspicious
{
  meta:
    description = "Shadow copy deletion or security tooling being disabled"
    severity = "critical"
  strings:
    $v1 = "vssadmin delete shadows" nocase
    $v2 = "wbadmin delete catalog" nocase
    $v3 = "bcdedit /set {default} recoveryenabled no" nocase
    $d1 = "Set-MpPreference -DisableRealtimeMonitoring" nocase
    $d2 = "netsh advfirewall set allprofiles state off" nocase
  condition:
    any of them
}
```

Structurally identical — `any of them` over a string list — but far more durable, because these strings are not names an attacker chose. They are **the command syntax the operating system requires**. To delete volume shadow copies you must invoke `vssadmin delete shadows`; there is no synonym. Renaming your tool does not help, because the string is not your tool's name, it is the API you are obliged to call.

That is the hierarchy worth internalising:

Rule keys on	Durability	Example
Tool name or author	Weakest — defeated by renaming	"mimikatz", "gentilkiwi"
Tool-specific syntax	Moderate — survives renaming, not rewriting	"sekurlsa::logonpasswords"
OS-required syntax	Strong — attacker cannot avoid it	"vssadmin delete shadows"
Structural requirement	Strongest — survives obfuscation	input source + execution primitive

Aim as far down that table as the target allows. Most rulesets are top-heavy because the top is easy to write, and that is why most rulesets are more fragile than their authors believe.

Metadata is a contract

One last property of the case study's rules, easy to overlook because it does not affect matching at all: every rule declares `severity` in its metadata, and the application reads it.

```
SEVERITY_BY_TAG = {
    "critical": "critical", "ransomware": "critical",
    "malware": "high", "webshell": "high", "exploit": "high",
    "suspicious": "medium", "pua": "low", "test": "info",
}

@staticmethod
def severity_of(meta, tags):
    declared = str(meta.get("severity", "")).lower().strip()
    if declared in SEVERITY_RANK:
        return declared
    for tag in tags:
        mapped = SEVERITY_BY_TAG.get(str(tag).lower())
        if mapped:
            return mapped
    return "medium"
```

Explicit `severity` wins; a tag mapping is the fallback; unlabelled rules default to medium. The consequence is that **the person who writes the rule sets its triage priority**, in the same file, at the same moment, with the threat fresh in mind — rather than an operator guessing weeks later from a rule name. The alternative, a severity table maintained separately from the rules, drifts out of sync within a month. Every time.

This is a small design decision with an outsized effect on how a ruleset ages, and it costs one line per rule.

PART THREE

Building the Sensor

Everything between a rule and an alert — and the four bugs that only appear once real files start arriving.

CHAPTER FIVE

Anatomy of a Detector

A detection engine is the easy part. Everything around it — deciding what to scan, when to scan it, what to record, and who to tell — is the system.

Starting from a script

Security Suite began as a single file of about ninety lines. It is worth looking at what that script got right before dismantling it, because the instincts behind it were sound:

- **It used `yara-python`, not `subprocess`.** Shelling out to the `yara` binary means a process spawn and a rules recompile per file. In-memory matching against a pre-compiled ruleset is orders of magnitude faster.
- **It wrote NDJSON.** One JSON object per line, appended. This is the correct format for an event log — streamable, greppable, and ingestible by every SIEM without a parser.
- **It attempted correlation.** Most first attempts at a file scanner emit only the file verdict. Reaching for authentication context is the right instinct, even though the implementation was broken in a way Chapter 7 dissects at length.
- **It was honest about its fallback.** When no rules file existed it compiled a test rule rather than pretending to be armed.

What it lacked was not cleverness. It lacked the accumulated scar tissue of having run against a real filesystem — which is precisely the material of the next two chapters.

Decomposition

The rebuilt system splits into six modules, each with one job:

Module	Responsibility	Why it is separate
<code>config.py</code>	Defaults, <code>config.json</code> loading, directory creation	Every other module takes configuration as an argument, so all of them are testable with a synthetic config.
<code>engine.py</code>	Compile rules, scan bytes, compute hash and entropy, map severity	Pure function of input bytes. No I/O policy, no scheduling, no state. Easy to test in isolation.
<code>telemetry.py</code>	Collect recent authentication failures from the OS	Platform-specific and permission-sensitive. Isolating it means the rest of the system does not care which OS it is on.
<code>watcher.py</code>	Decide what to scan and when; record outcomes	All the timing policy lives here — the settle logic, the rescan rules, the exclusions.
<code>store.py</code>	Persist events, hold recent	One place that owns durability and the

Module	Responsibility	Why it is separate
	history, publish to subscribers	live feed, so the monitor never talks to the HTTP layer.
<code>server.py</code>	HTTP API, static files, Server-Sent Events	Presentation. Removing it leaves a fully functional headless detector.

The load-bearing property here is that **the engine knows nothing about files arriving and the watcher knows nothing about YARA internals**. When the settling bug in Chapter 6 needed fixing, the change was confined to `watcher.py`. When the match-detail format changed, only `engine.py` moved. Detection systems accumulate special cases for years; module boundaries are what stop those special cases from becoming a single thousand-line function nobody will touch.

The engine: compile once, scan many

Rule compilation is expensive; matching is cheap. So compilation happens once at startup, and the compiled object is held and shared:

```
def reload(self) -> dict:
    """Recompile rules. Bad files are reported, not silently swallowed."""
    started = time.perf_counter()
    errors, sources = [], {}

    for path in self._rule_files():
        namespace = path.stem
        try:
            text = path.read_text(encoding="utf-8", errors="replace")
            yara.compile(source=text) # validate individually for a precise
error
        except (OSError, yara.Error) as exc:
            errors.append({"file": str(path), "error": str(exc)})
            continue
        sources[namespace] = text

    using_fallback = False
    if not sources:
        sources = {"builtin": FALLBACK_RULE}
        using_fallback = True

    compiled = yara.compile(sources=sources)
```

The detail worth stealing is the double compile. Each rule file is compiled **on its own first**, purely to find out whether it is valid, and only then are the good ones combined. Compile them all at once and a single syntax error in one file fails the whole batch with a message that may not even name the offending file — so an operator who adds a broken rule loses their entire ruleset and gets an unhelpful error. With per-file validation, one bad file is reported by name and the other twelve rules keep working.

FAIL LOUDLY, DEGRADE GRACEFULLY

These two principles look contradictory and are not. *Degrade gracefully*: one broken rule file must not take down detection. *Fail loudly*: the operator must know, in the interface, that a file is broken and which one.

The original script did the first without the second — it caught the compile error and quietly fell back to a dummy rule. A detector that has silently disarmed itself while continuing to report green is worse than one that has crashed, because nobody investigates a healthy-looking dashboard.

Enrichment at scan time

A match tells you a rule fired. It does not tell you what the file was. The scan path attaches identity and shape:

```
def scan_file(self, path: str) -> dict:
    stat = os.stat(path)
    if stat.st_size > self.max_file_bytes:
        return {"file_path": str(path), "skipped": "file larger than max_file_mb",
                "file_size": stat.st_size, "matches": [], "severity": None}

    started = time.perf_counter()
    with open(path, "rb") as handle:
        head = handle.read(1 << 20) # first MiB drives entropy + preview
    raw_matches = rules.match(filepath=str(path), timeout=60)
    result = self.result(str(path), head, raw_matches, started, stat.st_size)
    result["sha256"] = sha256_of(path)
    result["file_name"] = os.path.basename(path)
    result["modified"] = time.strftime("%Y-%m-%dT%H:%M:%S",
    time.localtime(stat.st_mtime))
    return result
```

Four pieces of enrichment, each earning its cost:

- **SHA-256** is the file's identity. It is what you paste into VirusTotal, what you search your estate for, and what tells you that the thing in `uploads/` is the same thing that appeared on a workstation last Tuesday. Without a hash, an alert refers to a path that may not exist tomorrow.
- **Size** bounds cost and provides context. A 43-byte web shell and a 40 MB one are different stories.
- **Entropy** over the first megabyte hints at packing or encryption. Computed on a bounded prefix, not the whole file, so it stays cheap.
- **Modification time** is the beginning of a timeline. Combined with the scan timestamp it tells you how long the file sat before anyone noticed.

Note the size guard runs *before* anything is read. A detector that tries to hash a 200 GB file because nobody bounded it is a detector that becomes an outage.

Recording the match, not just the verdict

The original script recorded matches as `[str(m) for m in matches]` — a list of rule names. Everything else YARA returns was discarded. What the analyst needs is the evidence:

```
for match in raw_matches:
    meta = dict(match.meta)
    tags = list(match.tags)
    strings = []
    for sm in match.strings:
        for instance in sm.instances[:5]:
            strings.append({
                "identifier": sm.identifier,
                "offset": instance.offset,
                "preview": printable(instance.matched_data),
            })
    matches.append({
        "rule": match.rule, "namespace": match.namespace, "tags": tags, "meta":
meta,
        "severity": self.severity_of(meta, tags),
        "description": meta.get("description", ""),
        "strings": strings[:12],
    })
```

The difference in triage is stark. `Ransom_Note_Template` tells the analyst a rule fired. `$a1` at `0x0`: "Your files have been encrypted" and `$b1` at `0x2b`: "bitcoin" tells them *what the file actually says*, and that is usually enough to reach a verdict without ever opening it — which matters, because opening suspicious files is how analysts become incidents.

Two bounds keep this from becoming a liability: at most five instances per string, at most twelve strings per match. A rule matching a string that occurs fifty thousand times should not produce a fifty-thousand-element JSON array. And `printable()` renders bytes as safe ASCII with dots for non-printables, so binary content cannot corrupt the log or the terminal rendering it.

The store: durability, history, and a live feed

`store.py` owns three concerns that look separate and are not: what gets written to disk, what stays in memory, and who gets told.

```
def add(self, event: dict, persist: bool = True) -> dict:
    event.setdefault("id", new_id())
    event.setdefault("timestamp", now_iso())
    event.setdefault("status", "new")
    event.setdefault("_epoch", time.time())
    with self._lock:
        if persist:
            try:
                with self.path.open("a", encoding="utf-8") as handle:
                    handle.write(json.dumps(event, default=str) + "\n")
            except OSError as exc:
                print("[-] Could not persist finding: " + str(exc))
        self._events.append(event)
        self._count(event)
        dead = []
```

```

        for sub in self._subscribers:
            try:
                sub.put_nowait(event)
            except queue.Full:
                dead.append(sub)
        for sub in dead:
            self._subscribers.remove(sub)
    return event

```

The `persist` flag is a deliberate policy decision. **Matches are written to disk; clean scans are not.** A directory with 50,000 benign files would otherwise produce 50,000 lines of `"verdict": "clean"` per sweep, and `findings.ndjson` would become an audit trail of every file ever seen rather than a log of things that need attention. Clean scans still live in the in-memory ring buffer, so the dashboard can show throughput — they simply do not consume disk forever.

The subscriber loop is the live feed. Each connected browser holds a bounded queue; a slow or dead client fills its queue and is dropped rather than blocking the scanner. This is the correct priority: a detector must never stall because a dashboard tab is unresponsive.

Why the ring buffer matters

`collections.deque(maxlen=2000)` bounds memory regardless of uptime. On startup, the tail of the NDJSON file is replayed into it:

```

with self.path.open("r", encoding="utf-8", errors="replace") as handle:
    tail = deque(handle, maxlen=self.history_limit)

```

`deque(file_object, maxlen=N)` reads the file and keeps only the last N lines, without holding the whole file in memory. It is a neat idiom and exactly right here: restarting the service does not empty the dashboard, but a 4 GB findings log does not blow up the process either.

Choosing not to use a framework

The HTTP layer is `http.server` from the standard library rather than FastAPI or Flask. That is unusual enough to justify.

A detection tool is often deployed onto a machine that is already having a bad day — an incident response context, a locked-down server, a host with no outbound internet to reach a package index. Every dependency is a thing that must be installed, version-pinned, audited, and kept patched. The suite has exactly one hard dependency, `yara-python`, because that one is doing irreplaceable work. The web framework was not: about 240 lines of `http.server` provides routing, JSON responses, static files, and Server-Sent Events.

This is not an argument against frameworks in general. It is an argument that **security tooling has a different dependency calculus than product code**, because it needs to run in degraded environments and because its supply chain is itself a target.

CHAPTER SIX

The Race Conditions Nobody Warns You About

Four bugs, all invisible in testing, all discovered by running the thing against a real filesystem. Each one is a category, not an incident.

This is the most useful chapter in the book, because none of these problems appear in tutorials. They appear the first week you run a scanner for real, and each one degrades detection silently.

Bug one: the half-written file

The naive monitor is the one everybody writes first:

```
seen_files = set(os.listdir(WATCH_DIRECTORY))
while True:
    time.sleep(2)
    current_files = set(os.listdir(WATCH_DIRECTORY))
    for file_name in current_files - seen_files:
        scan(os.path.join(WATCH_DIRECTORY, file_name))
    seen_files = current_files
```

A file appears in a directory listing the moment it is *created*, not when it is finished. Upload a 50 MB archive over a slow connection and the filename exists within milliseconds while bytes keep arriving for the next thirty seconds. The scanner sees a new name, scans what is there — perhaps 2 MB of a 50 MB file — finds nothing, marks it seen, and never looks again.

The failure is silent and total. No error, no warning, a clean verdict on a file that was never fully examined. In testing you will never see it, because test files are small and written instantly.

The fix: settle detection

A file is scanned only once its size and modification time have stopped changing for a configured interval:

```
signature = (stat.st_mtime, stat.st_size)
if self._scanned.get(path) == signature:
    continue # unchanged since the last scan

previous = self._pending.get(path)
if previous is None or previous[:2] != signature:
    # New or still being written: restart the settle timer.
    self._pending[path] = (stat.st_mtime, stat.st_size, now)
    continue
if now - previous[2] < self.cfg.settle_seconds:
    continue # not settled yet
```

```
self._pending.pop(path, None)
self._scanned[path] = signature
self.scan_and_record(path, trigger="monitor")
```

Two dictionaries do the work. `_pending` tracks files still changing, with the timestamp at which the current signature was first observed. `_scanned` records the signature at the moment of the last scan. A file moves from pending to scanned only when it has been stable across at least one full poll interval plus `settle_seconds`.

Because `_scanned` stores the signature rather than just the path, a **modified** file is rescanned automatically — its new size or mtime will not match the stored signature. The naive version, storing only names in a set, could never notice a file changing after its first appearance. That is a second silent failure hiding inside the first.

VERIFYING A TIMING FIX

Timing bugs need timing tests. The case study's test writes a file in two pieces with a deliberate pause:

```
f.write("harmless start ") — flush — sleep 0.5s — f.write("malware") — flush
```

Then it asserts that **exactly one** event exists for that file and that it is a match. A premature scan would produce a clean verdict on the first fragment; a double scan would produce two events. One event, correct verdict, is the only passing outcome.

Bug two: the self-scan feedback loop

This one is delightful, in the way that only bugs happening to someone else are delightful.

The detector writes alerts to `findings.ndjson`. Each alert contains the matched strings — the evidence, as designed in Chapter 5. If that file happens to sit inside a watched directory, the scanner sees a new file, scans it, and finds the ransom-note strings it helpfully recorded there a moment ago. It alerts. The alert is appended to the same file. The file changes, so it is rescanned. It now contains two alerts' worth of malicious strings.

The output during discovery is unmistakable:

```
[!] ALERT CRITICAL Ransom_Note_Template in ../ss_test/note.txt
[!] ALERT HIGH PowerShell_Encoded_Command in ../ss_test/ps.txt
[!] ALERT CRITICAL Demo_TestKeyword, PowerShell_Encoded_Command,
  Ransom_Note_Template in ../ss_test/findings.ndjson
[!] ALERT CRITICAL Demo_TestKeyword, PowerShell_Encoded_Command,
  Ransom_Note_Template in ../ss_test/findings.ndjson
```

Every alert generating a further alert, each one accumulating more matched strings than the last. Left running, it fills the disk.

The fix, and the principle

```
# Never scan our own output: findings.ndjson contains the strings that
# tripped the rules, so scanning it would alert on itself forever.
```

```
self._self_paths = {
    os.path.normcase(os.path.abspath(p))
    for p in (cfg.findings_log, cfg.triage_file)
}
```

Checked on every candidate, with `normcase` and `abspath` so that `./data/findings.ndjson` and `C:\...\Data\Findings.NDJSON` compare equal on a case-insensitive filesystem.

The general principle is larger than this bug: **a detector's own artifacts are part of its environment**. Log files, quarantine directories, temporary extraction folders, and the rule files themselves all contain the strings the detector hunts. Any of them inside a monitored path creates a loop. It is the security-tool equivalent of audio feedback, and the fix is the same — keep the output away from the input.

THIS IS NOT A HYPOTHETICAL

Default configuration puts the findings log in `data/` and watches `uploads/`, so the shipped configuration is safe. The loop appeared the moment a test pointed both at one temporary directory — which is exactly what an operator does when they set `watch_paths` to a project root to "scan everything".

Bug three: the file that vanishes mid-scan

During testing, a real PHP web shell was written to disk to exercise the web shell rules. The scan raised an error, and the file was gone.

Windows Defender had quarantined it between the directory sweep and the scan — a window of a few hundred milliseconds. The detector had found a new file, decided to scan it, and by the time it called `os.stat()` the file no longer existed.

This is not an edge case. On any endpoint with antivirus, the detector is in a race with another security product over the same file, and it will lose that race regularly. Files also get moved by the process that uploaded them, deleted by cleanup jobs, or locked by a backup agent.

The fix: distinguish absence from failure

```
try:
    result = self.engine.scan_file(path)
except (FileNotFoundError, PermissionError) as exc:
    # Normal on a live endpoint: antivirus quarantined the file, or it
    # was moved/locked between the sweep and the scan. Not an error.
    return self.store.add({
        "event_type": "scan",
        "file_path": str(path),
        "file_name": os.path.basename(path),
        "skipped": "unavailable at scan time (" + type(exc).__name__ + ")",
        "trigger": trigger,
    }, persist=False)
except Exception as exc:
    return self.store.add({
        "event_type": "error", "severity": "low", ...
    })
```

A missing or locked file is recorded as a skipped scan, not an error. Genuine faults — a corrupt ruleset, a YARA timeout, an out-of-memory condition — still surface as errors. The distinction matters because an error count that ticks up constantly from routine AV activity is an error count nobody reads, and the one real fault hides inside it.

THE LESSON BEYOND THE BUG

Your detector is not the only security product on the host, and it is not the privileged one. It shares the filesystem with antivirus, EDR, backup agents, and indexers — all of which move, lock, and delete files without telling it.

Design for a hostile filesystem: every file operation can fail, the file can disappear at any point, and none of that is exceptional. It is Tuesday.

Bug four: forgetting to forget

The last one is subtle. The naive monitor accumulates seen filenames forever. Delete a file and re-upload one with the same name, and it is never scanned again — its name is still in the set.

That is trivially exploitable if an attacker knows a filename that has already passed inspection, and it also breaks ordinary workflows where a build job writes `artifact.zip` repeatedly.

```
# Forget files that disappeared so a re-upload is scanned again.
for gone in [p for p in self._scanned if p not in seen_now]:
    self._scanned.pop(gone, None)
for gone in [p for p in self._pending if p not in seen_now]:
    self._pending.pop(gone, None)
```

Tracking state about a mutable world means the state must be reconciled with reality on every pass, in both directions — additions *and* removals. Most implementations handle additions only, because additions are what the feature is about.

The baseline decision

One design choice sits alongside these bugs. When the monitor starts and the watched directory already contains ten thousand files, should it scan them?

```
if not self.cfg.scan_existing_on_start:
    # Baseline: remember what is already there so we only alert on new work.
    for path, stat in self._iter_files():
        self._scanned[path] = (stat.st_mtime, stat.st_size)
```

The default is no: record what exists and alert only on what arrives afterwards. Scanning everything on every restart means a service restart produces ten thousand events and buries anything current. The opposite behaviour is available behind `--scan-existing` for the deliberate case — a fresh deployment onto an existing server, where you genuinely want to know what is already there.

Neither default is universally right. What matters is that it is a decision, documented and switchable, rather than an accident of implementation.

CHAPTER SEVEN

Telemetry, Time, and Correlation

A file verdict answers "is this bad?" Correlation answers "how did it get here, and what else happened?" Getting the second one wrong is easy, and the failure is invisible.

A bug worth studying in detail

The original script attached authentication context to every alert. Here is the function, in full:

```
def get_recent_auth_events(self):
    """Tails auth.log for failed logins within the LOOKBACK_MINUTES."""
    events = []
    now = datetime.now()

    try:
        with open(AUTH_LOG_PATH, "r") as f:
            for line in f:
                if "Failed password" in line or "Invalid user" in line:
                    # Basic check: is this line from 'today'?
                    events.append({
                        "timestamp": now.isoformat(),
                        "raw_log": line.strip(),
                        "type": "auth_failure"
                    })
        return events[-5:] # Return only the 5 most recent for brevity
    except FileNotFoundError:
        return [{"error": "auth.log not accessible"}]
```

The docstring says it filters to a lookback window. It does not. Read what the code does:

- It iterates **every line in the entire log**, which may span months.
- It stamps each matched line with `now` — the moment of the scan, not the moment of the event. The real timestamp, sitting at the start of the raw line, is never parsed.
- It returns the last five entries **in file order**, which is not the same as the five most recent in time, and is not filtered by age at all.

The consequence: an alert could arrive claiming five authentication failures inside a five-minute window that actually happened last March. Every event in the output carries an identical, fabricated timestamp.

WHY THIS CLASS OF BUG IS DANGEROUS

It produces *plausible* output. The alert has a telemetry section, it is populated, the timestamps look reasonable, and nothing errors. An analyst reading it draws a conclusion — "brute force preceded this upload" — that the data does not support.

A bug that produces no output gets found on day one. A bug that produces confidently wrong output can survive for years and actively misdirect an investigation. In security tooling, fabricated context is worse than no context.

Timestamps are the whole problem

Correlation is a claim about time: *these things happened near each other*. If your timestamps are wrong, every correlation built on them is wrong, and confidently so. So the rebuilt telemetry layer does one thing above all — it parses the real timestamp from each source.

Syslog, and the missing year

The traditional syslog format has an irritating property: no year.

```
Sep 11 14:23:01 host sshd[1234]: Failed password for invalid user root from
10.0.0.9
```

Month, day, time. Parse it against the current year and every log line in January, read in December, lands eleven months in the future:

```
@staticmethod
def _parse_syslog_time(line: str) -> datetime | None:
    match = SYSLOG_RE.match(line)
    if not match:
        return None
    month = MONTHS.get(match.group("mon"))
    if not month:
        return None
    now = datetime.now()
    hour, minute, second = (int(p) for p in match.group("time").split(":"))
    try:
        when = datetime(now.year, month, int(match.group("day")), hour, minute,
            second)
    except ValueError:
        return None
    # Syslog omits the year: a timestamp in the future means last year.
    if when > now + timedelta(days=1):
        when = when.replace(year=now.year - 1)
    return when
```

The one-day tolerance absorbs clock skew and timezone drift without misclassifying genuinely recent entries. The `ValueError` guard catches February 29th in a non-leap year — a real failure mode when the year is being guessed.

The file is also read from the end rather than the beginning, because only the tail can possibly fall inside a five-minute window:

```
size = os.path.getsize(path)
with open(path, "r", encoding="utf-8", errors="replace") as handle:
    handle.seek(max(0, size - 512 * 1024))
    if size > 512 * 1024:
        handle.readline()      # discard the partial first line
```

Seek to 512 KB from the end, discard the first (probably truncated) line, and read forward. On a multi-gigabyte auth log this is the difference between a correlation that costs microseconds and one that costs minutes — and a correlation slow enough to notice is one an operator will eventually disable.

The Windows Security log

Windows keeps authentication events in a structured event log rather than a text file, which is better in every way except discoverability. The events that matter:

Event ID	Meaning	Why it is worth correlating
4625	Failed logon	The primary brute-force and credential-stuffing signal.
4648	Logon with explicit credentials	Often lateral movement — a process using credentials other than the caller's.
4740	Account lockout	The loud end of a password-guessing campaign.
4771	Kerberos pre-auth failed	Domain-side equivalent of 4625; catches attempts against AD directly.
4776	Credential validation failed	NTLM validation failure, including local accounts.

Reading them is a backwards walk from the newest record, stopping as soon as an event falls outside the window:

```

handle = win32evtlog.OpenEventLog(None, "Security")
flags = win32evtlog.EVENTLOG_BACKWARDS_READ | win32evtlog.EVENTLOG_SEQUENTIAL_READ

while len(events) < self.max_events and scanned < 4000:
    records = win32evtlog.ReadEventLog(handle, flags, 0)
    if not records:
        break
    for record in records:
        scanned += 1
        generated = record.TimeGenerated
        when = datetime(generated.year, generated.month, generated.day,
                        generated.hour, generated.minute, generated.second)
        if when < cutoff:
            scanned = 4000      # everything older follows; stop the outer loop
            break
        event_id = record.EventID & 0x1FFFFFFF
        if event_id not in WATCHED_EVENT_IDS:
            continue

```

Two Windows-specific details. `EVENTLOG_BACKWARDS_READ` starts at the newest record, so the loop can stop early instead of reading the whole log. And `record.EventID & 0x1FFFFFFF` masks off the severity and facility bits that Windows packs into the high bits of the event ID — compare the raw value against 4625 and it will never match, which is a classic half-hour of confusion.

Permission as a first-class state

Reading the Security log requires a privilege an ordinary user does not hold. The naive handling is a bare `except` that returns an empty list, which is indistinguishable from "no failed logons occurred" — a dangerous ambiguity, since one means *nothing happened* and the other means *you are blind*.

```
except pywintypes.error as exc:
    # 5 = access denied, 1314 = SeSecurityPrivilege not held.
    denied = exc.winerror in (5, 1314)
    return {
        "status": "denied" if denied else "error",
        "source": self.source,
        "detail": ("Access denied reading the Security log - relaunch the suite as
"
                  "Administrator to enable logon correlation."
                  if denied else str(exc)),
        "events": [],
    }
```

Every telemetry result carries an explicit `status`: `ok`, `denied`, `unavailable`, or `error`. The dashboard renders it, and an alert produced while telemetry was denied says so in the finding itself. The analyst therefore knows the difference between *there were no failed logons* and *I could not see whether there were failed logons*, which is one of the more consequential distinctions in the whole system.

WINDOWS ERROR 1314

Access denied is error 5, and everybody handles it. Reading the Security log without the privilege actually returns **1314**, `ERROR_PRIVILEGE_NOT_HELD`, which a handler checking only for 5 reports as a generic error.

The difference is user-visible: a generic error message says "something went wrong", while a denial message says "run as Administrator" — one of which the operator can act on.

Choosing a window

The lookback window is the correlation's central parameter, and there is no universally correct value.

Window	Catches	Costs
1 minute	Only tightly coupled events — a logon immediately followed by an upload.	Misses a patient attacker; misses anything behind a queue or a slow transfer.
5 minutes	Interactive attack sequences: authenticate, upload, execute.	A busy server will show unrelated logon noise in every window.
1 hour	Slower campaigns and staged intrusions.	On any multi-user system, everything correlates with everything. The signal drowns.

Five minutes is the default because it approximates the timescale of a human driving a session. It is exposed as `lookback_minutes` precisely because the right answer depends on how busy the host is — on a single-purpose upload server, an hour may be perfectly informative; on a terminal server with two hundred users, even one minute will be noisy.

What a correlated alert actually claims

This deserves stating plainly, because correlation is routinely over-read.

When Security Suite attaches three failed logons to a web shell detection, it is claiming exactly one thing: **these events occurred within N minutes of each other on this host**. It is not claiming they are causally related. On a server exposed to the internet, failed SSH logons arrive continuously from background scanning; they will correlate with everything, including entirely benign events.

So correlation is a **prompt**, not a **conclusion**. It tells the analyst where to look next: same source IP as a recent successful logon? Same account that owns the upload directory? A burst against one account rather than scattered noise? Those questions turn temporal coincidence into a hypothesis. The tool supplies the coincidence; the analyst supplies the reasoning.

Tools that blur this — that present correlation as attribution — produce analysts who trust the machine's story over the evidence. Presenting telemetry as a labelled, timestamped, status-carrying list rather than a verdict is a deliberate choice to keep the human in the loop where the reasoning actually happens.

PART FOUR

Operating the Detector

Severity, attention, measurement, and the things that only break in production.

CHAPTER EIGHT

Severity, Triage, and the Economics of Attention

Analyst attention is the scarcest resource in security operations. Every design decision downstream of a rule is really a decision about how to spend it.

A detector that generates 500 alerts a day does not have 500 alerts of value. It has however many an analyst can actually judge — perhaps 40 — and 460 opportunities to train that analyst to stop reading. This chapter is about the machinery that decides which 40.

A severity scale that means something

Severity scales are usually invented in a meeting and then applied inconsistently forever, because nobody wrote down what the levels mean. Here is a definition worth stealing, phrased in terms of *response* rather than *badness*:

Level	Means	Expected response
critical	Confirmed malicious capability present.	Investigate now. Assume compromise until disproven. Web shells, credential dumpers, ransom notes, shadow copy deletion.
high	Strong indicator of attacker tradecraft; benign explanations exist but are uncommon.	Investigate this shift. Encoded PowerShell, download cradles.
medium	Suspicious pattern with plausible benign causes.	Review in batch. Packed executables, double extensions.
low	Weak signal; useful as corroboration, not on its own.	Do not alert. Query it when investigating something else.
info	Test and calibration rules. Confirms the pipeline works.	Never alert. Dashboard only.

The column that makes this usable is the third one. "Critical means very bad" is unfalsifiable and therefore unenforceable. "Critical means someone stops what they are doing" is a claim you can check against reality — and if your critical alerts do not actually cause anyone to stop what they are doing, they are not critical, whatever the metadata says.

THE SEVERITY INFLATION DEATH SPIRAL

Rule authors want their rules taken seriously, so they label them high. Soon most rules are high, so high stops meaning anything, so the next author labels theirs critical. Within a year everything is critical and the scale carries no information.

The defence is the response column. If every rule is critical, then by definition every alert requires immediate investigation — an obviously false claim, and one you can now point at in review.

Severity belongs to the rule

Chapter 4 established the mechanism; here is why it is an operational decision rather than an implementation detail.

The person writing a rule has the threat model in their head at that moment. They know whether this is a confirmed-bad indicator or a heuristic, what the false positive profile looks like, and what they would want to happen when it fires. That knowledge is at its peak while the rule is being written and decays immediately afterwards.

Storing severity anywhere other than the rule file guarantees drift. A separate mapping table, a database column, a dashboard configuration — each is a second thing that must be updated when a rule changes, and within a month someone will add a rule and forget. Putting `severity = "critical"` three lines above the condition makes the two impossible to separate: they move together, they get reviewed together, and they get deleted together.

The fallback chain matters too. Explicit metadata wins; a tag mapping catches rules imported from elsewhere; the final default is medium rather than info, so an unlabelled rule is visible rather than silently ignored. **Defaults should fail toward attention, not away from it.**

Triage states

An alert is not a notification. It is a small unit of work with a lifecycle, and a system without states is a system where every alert is permanently open:

State	Meaning
<code>new</code>	Nobody has looked. The only state that counts toward the open-alert metric.
<code>acknowledged</code>	A human has seen it and is working it. Prevents duplicate effort in a team.
<code>resolved</code>	Investigated and handled. Real, and dealt with.
<code>false_positive</code>	Investigated and wrong. This is a rule defect, not an analyst outcome.

That last row carries the weight. A false positive is feedback about detection content. If they are recorded as a disposition and never looked at again, the rule that produced them keeps producing them forever and nothing improves. The state exists so the tuning loop from Chapter 1 has an input — which is why the case study attaches an optional note when marking one, and persists it:

```
def set_status(self, event_id: str, status: str, note: str = "") -> dict | None:
    with self._lock:
        target = next((e for e in self._events if e.get("id") == event_id), None)
        if target is None:
            return None
        target["status"] = status
        target["triaged_at"] = now_iso()
        target["triage_note"] = note
        self._triage[event_id] = {"status": status, "at": target["triaged_at"],
        "note": note}
        self.triage_path.write_text(json.dumps(self._triage, indent=2),
        encoding="utf-8")
        return dict(target)
```

Triage state lives in a separate file from the findings log, and that separation is deliberate. `findings.ndjson` is **append-only and immutable** — the record of what the detector observed, which must never be rewritten. Triage is **mutable human judgement about** those events. Mixing them would mean rewriting an evidentiary log every time somebody clicks a button, which is exactly the property you do not want in something that might end up in an incident report.

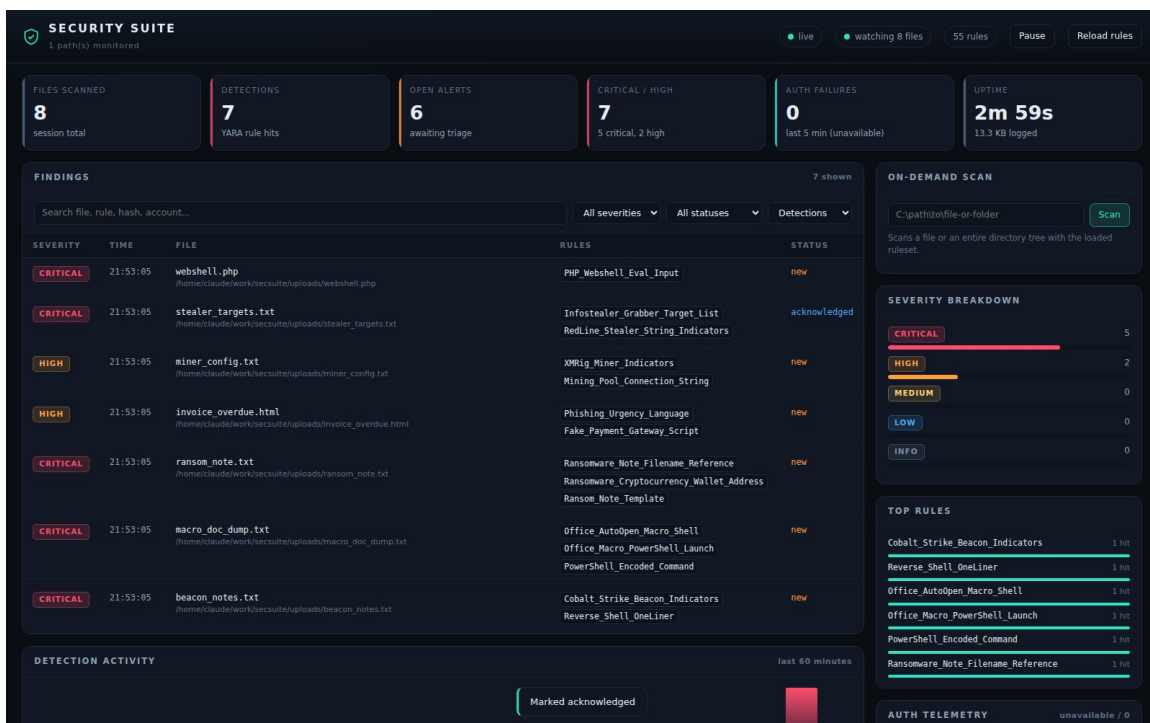


Figure 8.1 — A finding moved from new to acknowledged. The status column and the activity feed both update from the same triage call.

The only KPI that matters

Six numbers sit across the top of the dashboard: files scanned, detections, open alerts, critical/high count, correlated auth failures, uptime. Only one of them is a health metric.

Files scanned is a throughput number. It proves the sensor is alive and says nothing about whether it is effective. **Detections** measures how much your rules fire, which is a property of your ruleset and your traffic, not of your security posture — a spike may mean an attack or a new benign file type.

Open alerts is the real one. It is the size of the queue of things a human has not yet judged, and it is the number that tells you whether the system is sustainable. If it trends up, one of three things is true: you are under attack, your rules are too noisy, or you do not have enough analyst time. All three demand action, and none of the other five numbers will tell you.

A RULE OF THUMB

If open alerts exceed what one person can triage in one shift, the detector is no longer a control — it is a backlog generator. The fix is almost never "hire more analysts". It is to tune the rules that produce the most alerts with the fewest true positives.

The dashboard's *top rules* panel exists for exactly this. It ranks rules by hit count, and the rule at the top of that list on a quiet week is the rule to examine first.

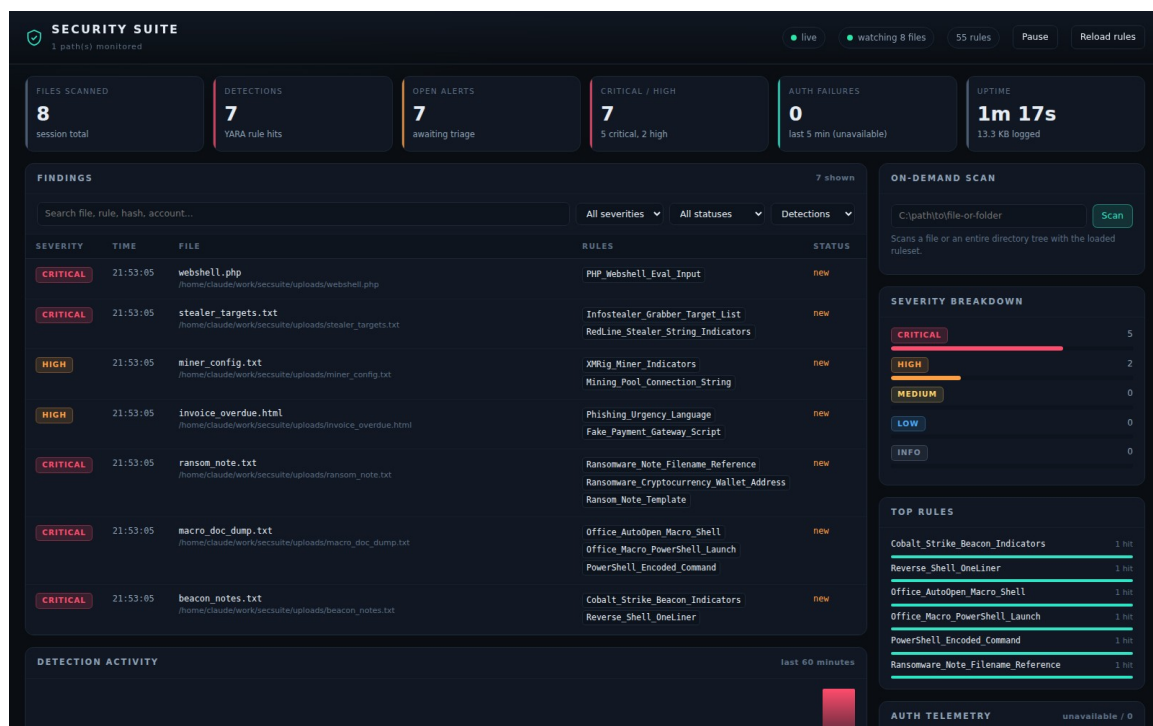


Figure 8.2 — The six KPIs, the findings table, and the severity breakdown, live against the ruleset shipped with this book.

Designing the alert for the reader

A triage decision is a reading task under time pressure. The interface should answer, in order: **how bad, what file, which rule, why did it match, what else was happening.**

The case study's detail view does this literally — verdict and severity first, then file identity with hash and entropy, then each matched rule with its description, then the matched strings with offsets, then the correlated telemetry with its status. An analyst reaches a verdict by reading down the page and stopping when they have enough, which is usually at the matched strings.

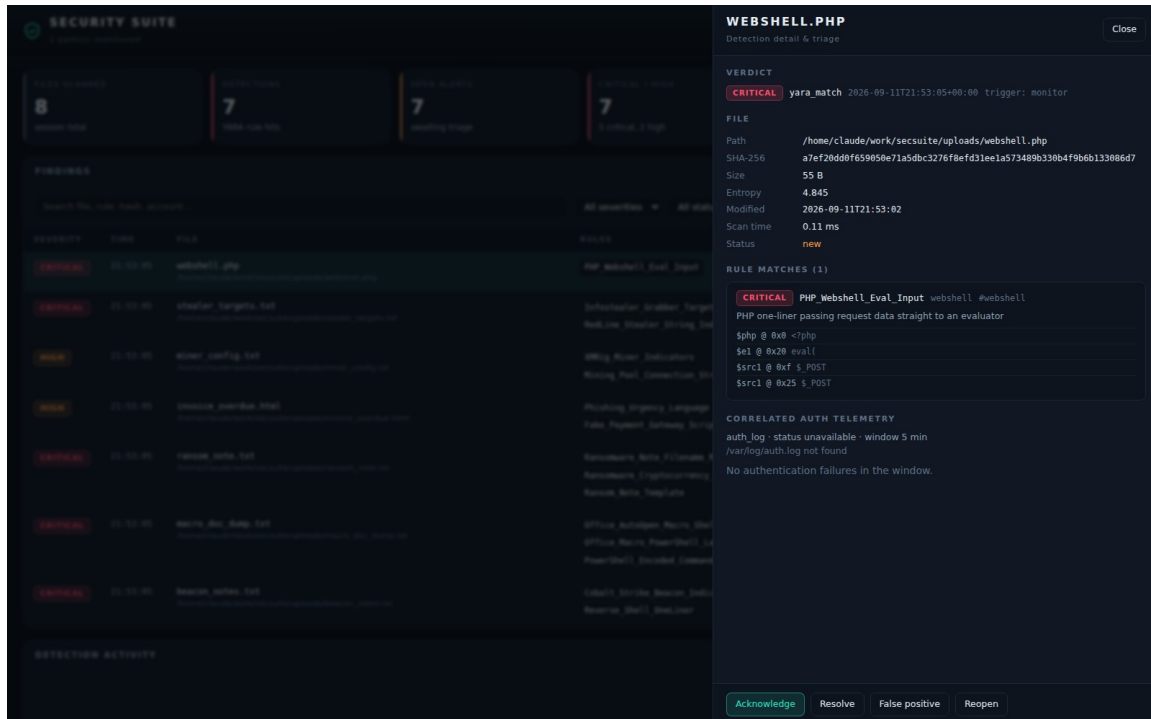


Figure 8.3 — The detail drawer for a PHP web shell: verdict, file identity, the matched rule, and the exact strings that fired, each with its byte offset.

The alternative — a table of rule names requiring the analyst to go and find the file themselves — moves the work from the machine to the human, and multiplies it by the number of alerts. Enrichment is computed once per detection; investigation is repeated per analyst per alert. That asymmetry is the entire argument for spending scan time on context.

What This Actually Stops: A Survey of Internet Crime

Severity and triage only mean something once you know what population of attacks they are sorting. The ruleset shipped with this book — fifty-five rules across twelve namespaces, listed in full in Appendix A — was built up category by category, in the same terms an incident responder uses to describe a case. This is the map.

Category	Rule file	What it stops
Server-side backdoors	webshell.yar	A PHP, ASPX, or JSP file dropped into an uploads directory that turns a web server into a remote command shell — the attack surface Chapter Two builds the whole book around.
Ransomware	ransomware.yar	Named-family strings (LockBit, Conti, Sodinokibi), ransom-note filenames, and a wallet address sitting next to extortion language.

Category	Rule file	What it stops
Credential theft	credential_theft.yar	Reading a browser's saved-password store, dumping the SAM hive, revealing a Wi-Fi key, or the API pair behind almost every user-mode keylogger.
Phishing and social engineering	phishing_social_engineering.yar	Credential-harvest pages impersonating a login flow, BEC wire-transfer language, fake-invoice lures, and QR-code redirection.
Malicious Office documents	office_macro_malware.yar	Auto-running macros that launch a shell or PowerShell, DDE field-code execution, and heavy Chr()-based obfuscation.
Unauthorized cryptomining	cryptominer.yar	XMRig command-line flags, a stratum mining-pool URI, and browser-based coin-mining scripts.
Command-and-control	c2_network.yar	Cobalt Strike beacon indicators, one-line reverse shells, DNS-tunneling query shapes, and .onion references paired with connection code.
Infostealers and RATs	info_stealer_rat.yar	A single sample enumerating multiple credential or wallet stores, RedLine and Discord-token-grabber strings, and RAT command sets (keylog, screenshot, webcam).
Reconnaissance and persistence	recon_persistence.yar	Scanner banners, BloodHound/SharpHound output, and the scheduled-task, Run-key, and WMI-subscription commands that make access survive a reboot.
Web and payment fraud	web_fraud_skimmer.yar	Magecart-style JavaScript skimmers reading card fields, fake payment gateways, and transparent clickjacking overlays on a checkout flow.
Windows host tradecraft	windows_threats.yar	Encoded PowerShell, download cradles, LSASS credential dumping, and defense-evasion commands such as shadow-copy deletion.

THE COMMON THREAD

Every rule in that table is an indicator or a behaviour, never a cryptographic signature of one exact malicious file. That is the distinction Chapter 4 draws between indicator rules and behaviour rules, and it is why the same twelve namespaces catch tooling that did not exist when the rules were written: a new infostealer that still enumerates wallet.dat and Login Data still matches `Infostealer_Grabber_Target_List`, because the rule was never about that stealer's name.

None of this is exhaustive, and it is not meant to be — Chapter 4's closing point stands: a rule is a bet about what won't change. The point of the severity scale in this chapter is to let an analyst triage across all twelve categories at once, without first having to decide by hand which kind of internet crime they are even looking at.

CHAPTER NINE

Measuring and Tuning a Ruleset

A ruleset that is never measured is a ruleset that is slowly getting worse, and you will not find out until it fails to catch something.

Precision and recall, in the terms that matter

Two numbers describe every detection, and they trade against each other:

- **Precision** — of the things that fired, what fraction were real? Low precision means wasted analyst time and, eventually, ignored alerts.
- **Recall** — of the real things, what fraction fired? Low recall means missed intrusions.

The hard asymmetry is that **you can measure precision and you cannot measure recall**. Precision falls out of your triage data: count the alerts, count how many were marked false positive, divide. Recall requires knowing about the attacks you missed, which by definition you do not. This is why the triage states from Chapter 8 are not bookkeeping — they are the only precision instrument you have.

Recall has to be estimated by other means: red team exercises, retrospective hunting over stored artifacts, replaying known-malicious samples through the detector, and post-incident review asking bluntly *why did this not fire?*

Testing detection content

Detection code needs tests as much as any other code, and it needs a specific kind: tests that assert on the *number* of events, not merely their presence.

```
# The webshell ruleset, verified in memory
for label, sample, rule in [
    ("php-eval", b"<?php @eval($_POST['c']); ?>", "PHP_Webshell_Eval_Input"),
    ("php-exec", b"<?php system($_GET['cmd']); ?>", "PHP_Webshell_Command_Exec"),
    ("jsp", b"<% Runtime.getRuntime().exec(request.getParameter('c')); %>",
    "JSP_Webshell"),
    ("aspx", b"<%@ Page %> Request.Form System.Diagnostics.Process'",
    "ASPX_Webshell"),
]:
    got = engine.scan_bytes(sample, label)
    names = [m["rule"] for m in got["matches"]]
    assert rule in names, (label, names)
    assert got["severity"] == "critical", got
```

Note that these samples are scanned **in memory**, never written to disk. That is not a stylistic choice — it is a direct consequence of Chapter 6's third bug. A real web shell written to a test directory gets quarantined by the host antivirus before the test can read it, so a disk-based test of web shell rules is flaky on precisely the machines where it matters most. Scanning bytes sidesteps the other security product entirely.

THE SAME THING HAPPENED TO THIS BOOK

An early draft of this chapter held that listing as a plain string in a source file. Windows Defender deleted the file mid-build — not the sample directory, not the detector, the *manuscript*.

The fix was to assemble the sample from fragments at build time, so no complete web shell literal ever exists on disk. If you write about detection content, you will eventually have to defend your own notes from your own antivirus.

Test the negative cases

Every ruleset needs benign samples that must *not* match, and they are more valuable than the positive cases. Positive tests tell you a rule works; negative tests tell you it has not quietly broadened into uselessness. The case study keeps `report_q3.txt` — ordinary business prose — in its sample set for exactly this reason, and the end-to-end test asserts the total detection count, so a rule that starts matching everything fails the suite loudly.

The tuning loop

When a rule produces a false positive, there are four responses, in rough order of preference:

7. **Add a required element.** The strongest fix. A rule firing on documentation about credential dumping gains a condition requiring PE structure, and the documentation stops matching while the binary still does.
8. **Narrow an existing element.** Add `fullword` to a short string, anchor a position, bound `filesize`. Cheap and often sufficient.
9. **Reduce the severity.** If the rule is genuinely a heuristic, say so. A medium-severity rule that batches for weekly review is far better than a high-severity one that gets ignored daily.
10. **Scope the rule.** Move it to a namespace applied only where it is precise. The credential-dumper rule is excellent on an FTP drop and useless in a folder of security research — same rule, different deployment.

Deleting the rule is a fifth option and sometimes the right one. A rule nobody acts on should not survive its first review.

Hot reload makes tuning cheap

The economics of tuning depend entirely on how long the loop takes. If changing a rule means restarting a service and losing state, tuning happens during maintenance windows — which means it does not happen.

```
elif route == "/api/rules/reload":
    info = self.ctx.engine.reload()
    self.ctx.store.broadcast({
        "event_type": "monitor", "timestamp": now_iso(),
        "message": "Reloaded " + str(info["rule_count"]) + " rules"
    })
```

```
self._json(info)
```

Edit the file, click the button, watch the rule count and the compile time update. The whole loop is a few seconds, and validation errors appear in the interface rather than in a log nobody is tailing. Making the right thing fast is most of making it happen.

Performance as a detection property

Scan time is recorded on every result, and it is not vanity instrumentation. A ruleset that grows slowly enough to notice will eventually be a ruleset that cannot keep up with file arrival — at which point the queue grows, scans fall behind, and detection becomes delayed rather than absent, which is harder to notice.

Typical figures from the case study on ordinary hardware: thirteen rules compile in roughly 25 ms; a small text file scans in one to three milliseconds; the seven-file sample directory completes in about 26 ms end to end. When a scan starts taking hundreds of milliseconds, the usual culprits are an unbounded regex, a rule without a `filesize` guard, or a `xor` modifier applied enthusiastically.

WHERE TO LOOK FIRST

Regular expressions and the `xor` string modifier dominate YARA performance problems. Both expand the search space enormously. Before reaching for either, ask whether two or three plain text strings combined in the condition would express the same idea — they almost always will, and they will run an order of magnitude faster.

CHAPTER TEN

Running It for Real

Deployment is where design decisions get audited by reality.

Privileges: ask for the minimum, degrade honestly

Security Suite needs read access to the directories it watches. That is all — for detection. Authentication correlation additionally requires the privilege to read the Windows Security log, which in practice means Administrator.

The design decision is that **the elevated capability is optional and its absence is visible**. Run unprivileged and file detection works completely; the telemetry panel shows `denied` with an explanation naming the fix. Run elevated and correlation activates.

The pattern generalises: when a tool needs elevation for one feature, do not require elevation for the whole tool. Requiring more privilege than necessary is how security tools become the largest attack surface on the host — a scanner reading untrusted attacker-supplied files while running as SYSTEM is a genuinely alarming combination, and one that the industry has repeatedly been embarrassed by.

The network boundary

The dashboard binds `127.0.0.1` and has no authentication. That combination is only safe because of the bind address, so the code defends it:

```
def _host_allowed(self) -> bool:
    """Block DNS-rebinding: only localhost names may talk to the API."""
    host = (self.headers.get("Host") or "").split(":")[0].strip("[]").lower()
    return host in ("localhost", "127.0.0.1", "::1", "")
```

Binding to loopback stops remote connections but does not stop **DNS rebinding**, where a malicious web page the operator visits resolves an attacker-controlled hostname to `127.0.0.1` and then issues requests to the local service from the browser. Those requests come from the local machine and pass the bind restriction. Validating the `Host` header rejects them, because the browser sends the attacker's hostname rather than a loopback name.

This is a small check that closes a real and frequently overlooked hole in local-only tools.

ON THE ON-DEMAND SCAN ENDPOINT

`POST /api/scan` will scan any path the running user can read, which is a broad capability — it is also the entire point of an on-demand scanner. The mitigation is not to cripple it but to keep the listener on loopback, validate `Host`, and never bind

0.0.0.0. If this ever needs to be reachable remotely, it needs authentication and path restrictions first, in that order.

Feeding a SIEM

NDJSON exists so that other systems can consume it without a parser. One JSON object per line, append-only, no enclosing array to close:

```
# Everything critical from the log
cat data/findings.ndjson | jq -c 'select(.severity=="critical")'

# Just the hashes, for an estate-wide search
jq -r '.sha256' data/findings.ndjson | sort -u

# Which rules fire most
jq -r '.rule_names[]' data/findings.ndjson | sort | uniq -c | sort -rn

# Ship it
tail -F data/findings.ndjson | your-log-shipper --stdin
```

The append-only property is what makes `tail -F` work as an ingestion strategy, and it is why triage state was kept in a separate file. A log that gets rewritten in place cannot be tailed reliably — the shipper either re-sends everything or loses its position.

What actually breaks

Symptom	Usual cause
Detections stop appearing, no errors	The monitor thread died on an exception the loop did not catch, or the watched path became unreachable — a network share that dropped. Check <code>last_sweep</code> and <code>last_error</code> in the sensor panel; a stale <code>last_sweep</code> is the tell.
Alerts for files nobody uploaded	Another process is writing into the watched directory — a backup agent, an indexer, or the detector's own output. See Chapter 6.
Scan errors climbing steadily	Antivirus quarantining files mid-scan. Expected on endpoints; these should appear as skipped scans rather than errors, and an exclusion for the watched path may be appropriate.
Telemetry permanently empty	Not elevated, or the Security log is full and not rotating. Check the telemetry <code>status</code> field before assuming quiet.
Memory growth over weeks	History limit set very high, or a rule matching a string with enormous instance counts. Both are bounded in the shipped configuration; both come back if the bounds are raised.
Dashboard blank, API responds	The browser cached an old asset, or an SSE connection failed silently. The stream indicator in the header shows <code>reconnecting</code> when this happens.

Extending it

Three directions, in increasing order of effort:

More rules. Drop a `.yar` file into `rules/` and reload. Nothing else changes; severity and namespaces are picked up automatically. This is where nearly all the value is, and it requires no code.

More telemetry sources. `telemetry.py` exposes one method, `recent()`, returning a dict with `status`, `source`, and `events`. A process-creation source, a DNS source, or a firewall source implementing the same shape would appear in the detail view with no changes elsewhere. The interface was kept deliberately narrow for this reason.

Different actions on detection. The suite is read-only by design. Moving a matched file to quarantine is straightforward to add and should be approached with the full weight of Chapter 1's warning: the moment a detection takes a destructive action, its precision requirements change completely, and a false positive stops being an inconvenience and becomes an outage.

Closing

The detector in this book is about 1,400 lines of Python. The rules are a hundred lines of YARA. None of it is technically sophisticated, and that is the point worth leaving with.

What separates a detector that works from one that merely runs is not algorithmic cleverness. It is having noticed that files are written incrementally. That antivirus will delete your evidence. That syslog omits the year. That a log file full of matched strings placed in a watched directory will alert on itself forever. That an analyst reading your alert at 3 a.m. needs the matched string, not the rule name.

Those are not insights you can derive from documentation. They come from running the thing, watching it fail in a way you did not predict, and fixing the category rather than the instance. That process — not the rules, not the dashboard — is detection engineering.

| *The rule is the easy part. Everything that happens after it fires is the job.*

— the whole book, compressed

APPENDICES

Reference

The complete ruleset, the HTTP API, every configuration key, and the command line.

APPENDIX A

The Complete Ruleset

Fifty-five rules across twelve namespaces, each named by its source file. Severity is declared in rule metadata and drives triage priority in the dashboard.

demo.yar — pipeline verification

Rule	Severity	Catches
Demo_TestKeyword	info	A literal test keyword. Smoke test for the whole pipeline.
EICAR_Test_File	low	The standard EICAR anti-malware test string. Harmless by design.
High_Entropy_Executable	medium	A PE file whose body exceeds 7.2 bits per byte — packed, encrypted, or compressed. Requires the math module.

webshell.yar — server-side backdoors

Rule	Severity	Catches
PHP_Webshell_Eval_Input	critical	PHP combining an evaluation primitive with request input (<code>\$_POST</code> , <code>\$_GET</code> , <code>\$_REQUEST</code> , <code>php://input</code>).
PHP_Webshell_Command_Exec	critical	PHP combining a shell function (<code>shell_exec</code> , <code>passthru</code> , <code>proc_open</code> , <code>popen</code>) with request input.
ASPX_Webshell	critical	An ASP.NET page directive plus a request accessor plus an execution primitive.
JSP_Webshell	critical	JSP passing <code>request.getParameter</code> to a runtime execution call.

windows_threats.yar — host tradecraft

Rule	Severity	Catches
PowerShell_Encoded_Command	high	PowerShell invoked with an encoded command flag or <code>FromBase64String</code> .
PowerShell_Download_Cradle	high	A download method (<code>DownloadString</code> , <code>Invoke-WebRequest</code> , <code>Net.WebClient</code>) paired with an execution method (<code>Invoke-Expression</code> , <code>IEX</code>).
Credential_Dumper_Indicators	critical	Strings from LSASS credential dumping tooling. Name-based; see Chapter 4 on its limits.

Rule	Severity	Catches
Defense_Evasion_Commands	critical	Shadow copy deletion, recovery disabling, real-time protection or firewall being turned off. OS-required syntax, so highly durable.
Ransom_Note_Template	critical	Encryption-notice language combined with a payment or contact indicator.
Suspicious_Double_Extension	medium	Filenames masquerading with a double extension, such as .pdf.exe.

ransomware.yar — encryption-for-extortion indicators

Rule	Severity	Catches
Ransomware_LockBit_String_Indicators	critical	Strings associated with the LockBit family.
Ransomware_Conti_String_Indicators	critical	Strings associated with the Conti family.
Ransomware_Sodinokibi_REvil_String_Indicators	critical	Strings associated with REvil / Sodinokibi.
Ransomware_Note_Filename_Reference	critical	Text referencing a canonical ransom-note filename.
Ransomware_Cryptocurrency_Wallet_Address	high	A BTC or Monero address next to extortion language.

credential_theft.yar — lifting secrets already on the host

Rule	Severity	Catches
Browser_Credential_Store_Access	high	Code paths reading a browser's saved-password database.
SAM_Registry_Hive_Dump_Command	critical	A command copying the SAM/SYSTEM hives for offline cracking.
WiFi_Saved_Password_Dump_Command	medium	netsh revealing a stored Wi-Fi key in cleartext.
Browser_Session-Token_Theft	high	Code reading a chromium leveldb path used by token grabbers.
Keylogger_API_Combination	high	The SetWindowsHookEx / GetAsyncKeyState pair behind most keyloggers.

phishing_social_engineering.yar — the lure itself

Rule	Severity	Catches
Phishing_Credential_Harvest_Form	high	A password field combined with brand-impersonation language.
Phishing_Urgency_Language	medium	Stock urgency phrasing used to short-circuit judgment.
Business_Email_Compromise_Wire_Request	high	Language typical of a BEC wire-transfer or gift-card ask.
Fake_Invoice_Attachment_Lure	medium	Filename or body text staging a fake overdue-invoice lure.
QR_Code_Phishing_Reference	medium	Body text steering a reader to scan a QR code to “reauthenticate”.

office_macro_malware.yar — Office documents as a delivery stage

Rule	Severity	Catches
Office_AutoOpen_Macro_Shell	critical	A VBA macro that runs on open and spawns a shell.
Office_Macro_PowerShell_Launch	critical	A macro handing off to an encoded PowerShell command.
Office_Macro_WMI_Persistence	high	A macro using WMI to spawn a process, bypassing normal logging.
Office_DDE_Field_Code	high	A DDEAUTO field code that executes on open without any macro.
Office_Macro_Heavy_Obfuscation	medium	VBA relying heavily on character-code reassembly.

cryptominer.yar — unauthorized coin mining

Rule	Severity	Catches
XMRig_Miner_Indicators	high	Command-line flags and pool protocol strings from XMRig-based miners.
Browser_Coin_Mining_Script	medium	An in-browser JavaScript miner embedded in a web page.
Mining_Pool_Connection_String	medium	A stratum mining-pool URI embedded in a file.

c2_network.yar — the phone-home half of an intrusion

Rule	Severity	Catches
Cobalt_Strike_Beacon_Indicators	critical	Strings commonly present in a Cobalt Strike beacon payload or config.
Generic_HTTP_Beacon_User-Agent	medium	A stale, hardcoded legacy User-Agent reused by beacon frameworks.
Reverse_Shell_OneLiner	critical	A one-line reverse shell in a common shell/python/bash form.
DNS_Tunneling_Query_Pattern	medium	A DNS label long and high-entropy enough to be a tunnel.
TOR_Onion_C2_Reference	medium	A .onion address referenced alongside connection code.

info_stealer_rat.yar — commodity stealers and RATs

Rule	Severity	Catches
RAT_Persistence_Register_Run_Key	high	A payload path under AppData registered as a Run key.
Infostealer_Grabber_Target_List	critical	One sample enumerating multiple distinct credential/wallet stores.

Rule	Severity	Catches
RedLine_Stealer_String_Indicators	critical	Strings associated with the RedLine stealer family.
Discord_Token_Grabber_Indicators	high	Code that reads and exfiltrates a Discord auth token.
Remote_Access_Trojan_Command_Set	critical	A cluster of remote-control command verbs typical of a RAT.

recon_persistence.yar — mapping the network and staying put

Rule	Severity	Catches
Network_Scanner_Tool_Banner	medium	Banner/usage strings from a common network scanning tool.
AD_Enumeration_Tool_Strings	high	Strings from a common Active Directory attack-path tool.
Scheduled_Task_Persistence_Command	high	A command installing a scheduled task for persistence.
Registry_Run_Key_Persistence_Command	high	A command-line write to a Run/RunOnce autostart key.
WMI_Event_Subscription_Persistence	critical	A WMI permanent event subscription used as fileless persistence.

web_fraud_skimmer.yar — client-side fraud in a web front end

Rule	Severity	Catches
JS_Card_Skimmer_Exfil	critical	JavaScript reading card-form fields and shipping them off-host.
Magecart_Skimmer_Indicators	critical	Function/variable names reused across Magecart-style skimmers.
Fake_Payment_Gateway_Script	high	A page impersonating a payment brand while posting elsewhere.
Clickjacking_Transparent_Overlay	medium	A transparent, absolutely-positioned iframe over a checkout/login flow.

APPENDIX B

HTTP API Reference

All endpoints are loopback-only and reject requests whose `Host` header is not a loopback name. Responses are JSON unless noted.

Method	Endpoint	Purpose
GET	/api/state	Stats, monitor status, engine info, telemetry, and

Method	Endpoint	Purpose
		effective configuration in one call. The dashboard's periodic refresh.
GET	/api/findings	Filtered findings. Query parameters: <code>severity</code> , <code>status</code> , <code>type</code> , <code>q</code> (full-text), <code>limit</code> .
GET	/api/rules	Loaded rules with namespaces and tags, plus any compile errors by filename.
GET	/api/telemetry	Recent authentication failures. <code>force=1</code> bypasses the result cache.
GET	/api/stream	Server-Sent Events. Emits <code>hello</code> on connect, <code>event</code> per finding, <code>stats</code> every two seconds.
POST	/api/scan	<code>{"path": "..."} — scan a file or directory tree on demand.</code>
POST	/api/monitor	<code>{"action": "pause"}</code> or <code>{"action": "resume"}</code> .
POST	/api/rules/reload	Recompile the ruleset without restarting. Returns the new engine info.
POST	/api/triage	<code>{"id": "...", "status": "...", "note": "..."} — set a finding's triage state.</code>

Event shape

```
{
  "id": "a3f19c2b8e04",
  "timestamp": "2026-09-11T14:40:47-05:00",
  "event_type": "yara_match",
  "severity": "critical",
  "verdict": "malicious",
  "trigger": "monitor",
  "file_path": "C:\\...\\uploads\\README_RESTORE.txt",
  "file_name": "README_RESTORE.txt",
  "file_size": 97,
  "sha256": "2f20176c2252fac5e23e2f6e4221708af75db0b8407beb24c9fa1ec1b21e0435",
  "entropy": 4.4,
  "scan_ms": 1.21,
  "rule_names": ["Ransom_Note_Template"],
  "matches": [{
    "rule": "Ransom_Note_Template",
    "namespace": "windows_threats",
    "tags": ["malware"],
    "severity": "critical",
    "description": "Text that reads like a ransom note left beside encrypted files",
    "strings": [
      {
        "identifier": "$a1",
        "offset": 0,
        "preview": "Your files have been encrypted"
      },
      {
        "identifier": "$b1",
        "offset": 40,
        "preview": "bitcoin"
      }
    ]
  }],
}
```

```
"telemetry": {  
  "status": "denied",  
  "source": "windows_security_log",  
  "window_minutes": 5,  
  "count": 0,  
  "detail": "Access denied reading the Security log - relaunch as  
Administrator."  
},  
  "status": "new"  
}
```

APPENDIX C

Configuration Reference

Defaults live in `securitysuite/config.py`. To override, create `config.json` beside `run.py` containing only the keys you want to change; unknown keys are reported and ignored.

```
{
  "watch_paths": ["C:\\inetpub\\uploads", "D:\\ftp\\incoming"],
  "lookback_minutes": 10,
  "poll_interval": 1.0,
  "max_file_mb": 128,
  "port": 8787
}
```

Key	Default	Meaning
<code>watch_paths</code>	<code>["uploads"]</code>	Directories to monitor. Created if absent.
<code>recursive</code>	<code>true</code>	Include subdirectories.
<code>poll_interval</code>	<code>2.0</code>	Seconds between directory sweeps.
<code>settle_seconds</code>	<code>1.0</code>	A file must stop changing for this long before it is scanned. See Chapter 6.
<code>max_file_mb</code>	<code>64</code>	Files above this size are recorded as skipped, not scanned.
<code>ignore_suffixes</code>	<code>.tmp .part .crdownload .swp</code>	Extensions never scanned — in-progress download markers.
<code>scan_existing_on_start</code>	<code>false</code>	Whether to scan files already present at startup.
<code>rules_dir</code>	<code>rules</code>	Directory searched recursively for <code>.yar</code> and <code>.yara</code> files.
<code>lookback_minutes</code>	<code>5</code>	Telemetry correlation window.
<code>auth_log_path</code>	<code>/var/log/auth.log</code>	Linux and macOS only; ignored on Windows.
<code>telemetry_cache_seconds</code>	<code>15.0</code>	How long a telemetry result is reused before re-querying.
<code>max_telemetry_events</code>	<code>25</code>	Cap on correlated events attached to one finding.
<code>findings_log</code>	<code>data/findings.ndjson</code>	Append-only alert log. Keep it outside every watched path.
<code>trriage_file</code>	<code>data/trriage.json</code>	Mutable triage state, deliberately separate from the log.
<code>history_limit</code>	<code>2000</code>	In-memory ring buffer size; also how many log lines are replayed at startup.

Key	Default	Meaning
host / port	127.0.0.1 / 8787	Dashboard bind address. Do not bind a routable interface.

APPENDIX D

Command Line and Recipes

Installing It

The only hard requirement is yara-python. There is no web framework to install: the dashboard server is `http.server` from the standard library, and the front end is plain HTML, CSS, and JavaScript served as static files. On Windows, `pywin32` is an optional second dependency that unlocks reading the Security event log for logon-failure correlation; without it, detection still runs at full strength and the telemetry panel simply reports itself as unavailable.

```
git clone https://github.com/gocar112/security-suite-dashboard.git
cd security-suite-dashboard
pip install -r requirements.txt
python run.py
```

That last command opens `http://127.0.0.1:8787` in a browser automatically and starts watching the `uploads/` directory. Nothing here needs a build step or a package to compile — clone it, install the one required dependency, and run it.

ON WINDOWS, RUN AS ADMINISTRATOR

Detection works fully either way, but reading the Security event log for logon-failure correlation needs elevation. Without it, the dashboard's telemetry panel shows denied with an explanation rather than failing silently — see Chapter 7 for what that correlation is actually claiming.

Invocation

```
python run.py                                # monitor + dashboard, opens a browser
python run.py --watch D:\ftp --watch E:\inbox
python run.py --port 9000 --no-browser
python run.py --scan C:\Users\me\Downloads    # one-shot scan, JSON to stdout, exit
python run.py --headless                     # monitor only, no web server
python run.py --scan-existing                 # also scan files already present at
startup
python run.py --rules D:\team-rules           # alternate rules directory
```

Verifying an installation

The sample files each trip exactly one rule, which makes them a fast end-to-end check:

```
copy samples\README_RESTORE.txt uploads\
```

A critical `Ransom_Note_Template` finding should appear in the dashboard within about two seconds, without a refresh.

Sample	Rule	Severity
<code>README_RESTORE.txt</code>	<code>Ransom_Note_Template</code>	critical
<code>cleanup_commands.txt</code>	<code>Defense_Evasion_Commands</code>	critical

Sample	Rule	Severity
dump_notes.txt	Credential_Dumper_Indicators	critical
task_setup.log	PowerShell_Encoded_Command	high
update_helper.txt	PowerShell_Download_Cradle	high
mail_body.txt	Suspicious_Double_Extension	medium
report_q3.txt	<i>(nothing — clean scan)</i>	—

WHY THE SAMPLES ARE PLAIN TEXT

A genuine .php web shell or PE sample written to disk is quarantined by antivirus before the scanner can read it — the failure described in Chapter 6. Text samples survive, so the smoke test is reliable on a protected endpoint. The web shell rules are verified in memory instead.

Querying the log

```
# Critical findings only
jq -c 'select(.severity=="critical")' data/findings.ndjson

# Unique hashes for an estate-wide search
jq -r '.sha256' data/findings.ndjson | sort -u

# Rule hit counts, most frequent first
jq -r '.rule_names[]' data/findings.ndjson | sort | uniq -c | sort -rn

# Findings that had correlated authentication failures
jq -c 'select(.correlated_auth_failures > 0)' data/findings.ndjson
```

Further reading

- **YARA documentation** — yara.readthedocs.io. The module reference, particularly `pe` and `math`, repays a careful read.
- **MITRE ATT&CK** — attack.mitre.org. Useful for mapping rules to technique IDs, which makes coverage gaps visible.
- **The Sigma project** — the same reasoning applied to log events rather than files. Chapter 4's capability-plus-context pattern transfers directly.
- **The repository** — <https://github.com/gocar112/security-suite-dashboard/tree/main>. Every listing in this book is taken from it.